

SEMANA 02

HTML y HTML5

para el desarrollo web

«HTML no es código fuente: es un contrato semántico entre el autor del documento y el ecosistema de agentes que lo interpretan.»

— Ian Hickson, editor de la especificación HTML5

Hoja de ruta de la sesión

Lo que recorreremos en las próximas 4 horas de contacto docente

01

Historia y evolución

De SGML a HTML Living Standard.
Por qué entender la historia
ayuda a entender el lenguaje.

02

Fundamentos y estructura

Tags, atributos, contenido.
DOCTYPE, <html>, <head>, <body>.
Validación en W3C.

03

Semántica de HTML5

header, nav, main, article,
section, aside, footer.
<article> vs <section>.

04

Formularios y validación

13 nuevos tipos de <input>.
required, pattern, min/max.
Validación nativa sin JS.

05

Multimedia y APIs

<video>, <audio>, <canvas>,
Geolocation, Web Storage,
Drag-and-Drop.

06

XML y formatos de datos

El ecosistema XML.
XML vs JSON vs YAML.
Quándo usar cada uno.

Resultado de aprendizaje

Lo que el estudiante debe ser capaz de hacer al cierre de esta semana

Construir documentos HTML5 estructuralmente correctos, semánticamente significativos y **validables ante el W3C**, integrando formularios con validación nativa, contenido multimedia accesible, APIs nativas del lenguaje y entendiendo el lugar de XML en el ecosistema de marcado.

Articulación con la asignatura

RAU 1	Construir interfaces web semánticas, accesibles y responsivas con HTML5, CSS3 y JavaScript siguiendo W3C y WCAG 2.1.
Unidad I	Cliente — HTML5 + CSS3 + JavaScript (Semanas 1–4)
SWEBOK	KA03 Diseño de software · KA04 Construcción de software · KA10 Calidad

De SGML a HTML Living Standard

35 años de evolución del lenguaje de la web



Tres etapas clave: (a) fundacional 1991–1999 • (b) transición XML 2000–2008 (fracasó) • (c) era HTML5 desde 2008. Desde 2019, **Living Standard**: el lenguaje se actualiza continuamente sin saltos de versión.

Por qué fracasó la transición XML (XHTML)

La web real era demasiado tolerante a errores como para acatar XML estricto

XHTML (2000–2008)

Sintaxis XML estricta:

- Toda etiqueta debe cerrarse.
- Atributos siempre con comillas.
- Etiquetas en minúsculas obligatoriamente.
- Cualquier error de sintaxis hace que el navegador muestre la "Pantalla amarilla de la muerte".

Problema:

miles de millones de páginas web ya escritas con HTML "sucio" pero funcional. Los navegadores tuvieron que seguir tolerándolo.

HTML5 Living Standard (desde 2008)

Pragmatismo recuperado:

- Permite etiquetas no cerradas (en algunos casos).
- Mayúsculas/minúsculas tolerantes.
- Algoritmo de parsing único y predecible.
- Compatibilidad retroactiva con HTML 4.01.

Lección histórica:

los estándares exitosos no son los más "limpios", sino los que respetan la realidad existente y evolucionan incrementalmente.

Estructura mínima de un documento HTML5

Los cinco elementos imprescindibles que todo documento debe contener

plantilla.html

```
<!DOCTYPE html>
<html lang="es-EC">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width,
    initial-scale=1">
  <title>Mi primera página</title>
</head>
<body>
  <h1>Hola UTEQ</h1>
  <p>Contenido visible.</p>
</body>
</html>
```

1

<!DOCTYPE html>

Indica al navegador: "este documento es HTML5". Va siempre en la primera línea.

2

<html lang="es-EC">

Idioma del documento. Crítico para lectores de pantalla y motores de búsqueda.

3

<meta charset="UTF-8">

Codificación de caracteres. Sin esto, las tildes y la "ñ" se rompen.

4

<meta viewport>

Habilita el diseño responsive. Sin esto, en móvil se ve la página minúscula.

5

<title>...</title>

Título de la pestaña del navegador. Aparece también en Google.

Anatomía de un elemento HTML

Los tres pilares: etiquetas (tags), atributos y contenido

```
<a href="https://uteq.edu.ec"> UTEQ  
</a>
```

Etiqueta de apertura
(define el tipo de elemento: enlace)

Atributo
(nombre + valor entre comillas)

Contenido
(texto visible para el usuario)

Caso especial: elementos VOID

Algunos elementos no tienen contenido y no se cierran: `<img>`, `<br>`, `<hr>`, `<meta>`, `<input>`, `<link>`.

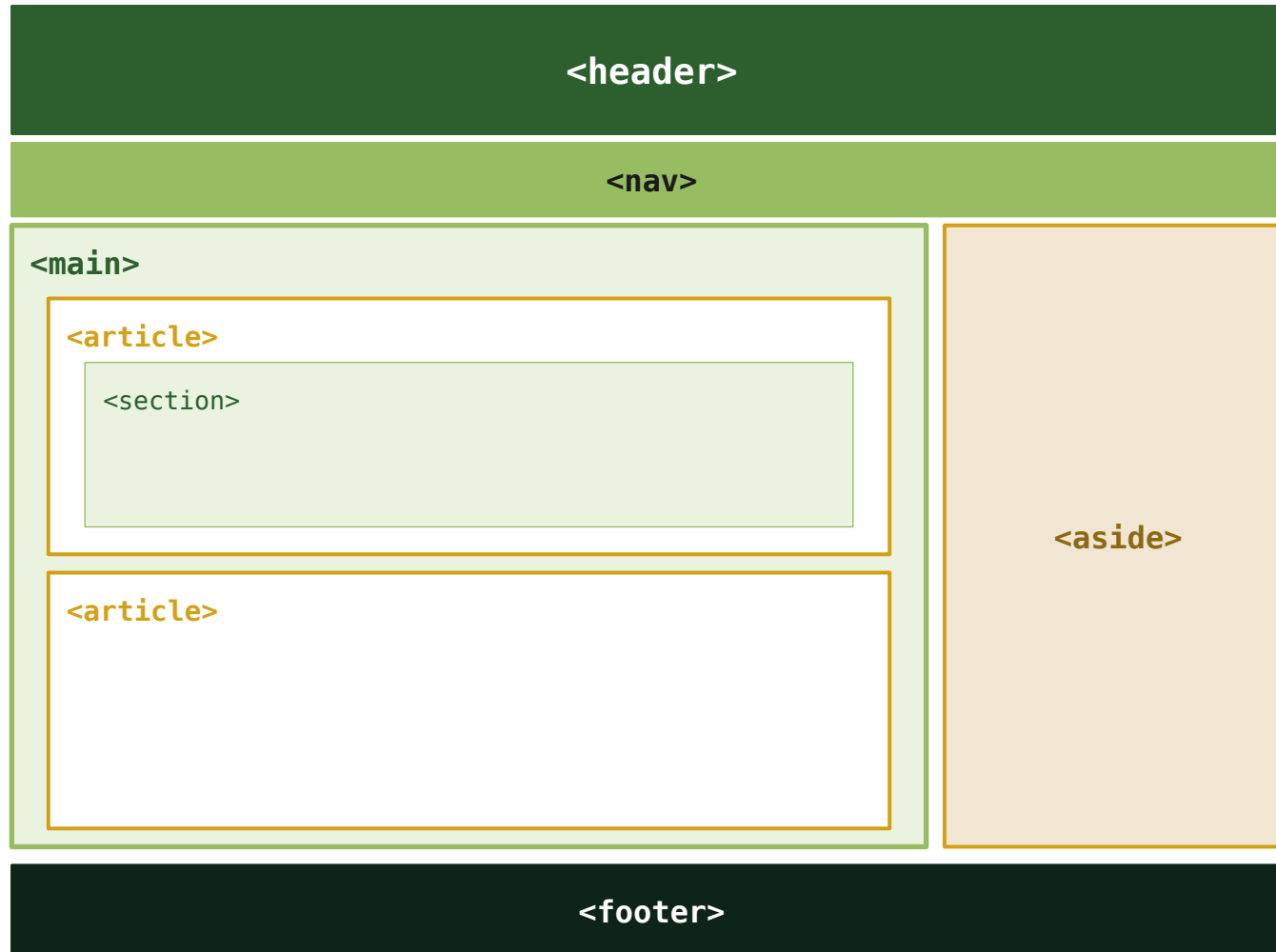
Se escriben con sintaxis auto-contenida:

```

```

Elementos semánticos de HTML5

Layout semántico vs. layout con <div> genéricos



`<header>`

Cabecera de la página o de una sección

`<nav>`

Bloque de navegación principal

`<main>`

Contenido principal (uno por documento)

`<article>`

Contenido autónomo y reutilizable

`<section>`

Agrupación temática con encabezado

`<aside>`

Contenido relacionado pero secundario

`<footer>`

Pie de página o sección

`<figure>`

Imagen o contenido autocontenido

`<time>`

Fecha u hora legible por máquinas

<article> vs <section>

La confusión más común — la "prueba RSS" la resuelve

<article>

*¿Tendría sentido este bloque
si se publicara solo, fuera de su contexto?*

Ejemplos canónicos:

- Entrada de blog
- Noticia
- Comentario en un foro
- Tarjeta de producto
- Tweet o post de red social
- Reseña de un libro o película

Test mental:

Si lo extraigo y lo pongo en un feed RSS o lo comparto en redes... ¿sigue teniendo sentido por sí mismo? Si la respuesta es Sí, es un <article>.

<section>

*Agrupación temática que SOLO tiene
sentido como parte del documento mayor.*

Ejemplos canónicos:

- Sección "Estudios" de un currículum
- Capítulo de un libro
- Sección "Características" en una landing page
- Subdivisiones internas dentro de un <article>
- Bloque de "Artículos relacionados"

Regla obligatoria:

Todo <section> debe tener un encabezado <h2>...<h6>. Si no necesita encabezado, probablemente debas usar <div>.

Ejemplo real: estructura de un blog

Cada entrada es un `<article>`; el bloque "Relacionados" es una `<section>`

```
<main>
  <h1>Blog de Aplicaciones Web - UTEQ</h1>
  <!-- ARTICLE: entrada que puede publicarse en RSS independiente -->
  <article>
    <header>
      <h2>Por qué HTML5 cambió el desarrollo web</h2>
      <p>Por <a href="/autor/jdoe">Juan Doe</a>
        el <time datetime="2026-05-04">4 mayo 2026</time></p>
    </header>
    <p>Introducción al artículo...</p>
    <!-- SECTION dentro de article: agrupación temática interna -->
    <section>
      <h3>Elementos semánticos nuevos</h3>
      <p>HTML5 introdujo header, nav, main...</p>
    </section>
  </article>
  <!-- SECTION: agrupación que NO tendría sentido fuera del blog -->
  <section aria-label="Artículos relacionados">
    <h2>También te puede interesar</h2>
    <ul><li><a href="/post/1">Una guía rápida de Flexbox</a></li></ul>
  </section>
</main>
```

BLOQUE 04

Formularios HTML5

y validación nativa

13 nuevos tipos de <input> · validación sin JavaScript · teclados virtuales optimizados

Nuevos tipos de <input> de HTML5

Tres ventajas: validación automática + interfaces especializadas + teclado móvil correcto

Tipo	Uso	Validación / efecto	Tipo	Uso	Validación / efecto
email	Correo electrónico	x@y.z + teclado con @ en móvil	time	Hora	Selector horario nativo
tel	Teléfono	Teclado numérico (use pattern)	datetime-local	Fecha y hora local	Calendario + reloj
url	URL completa	Formato URL + tecla .com	month	Año y mes	Selector específico de mes
number	Valor numérico	min / max / step	week	Semana del año	Selector específico de semana
range	Selector deslizante	min / max (visual, sin teclado)	color	Color hexadecimal	Paleta del sistema operativo
date	Fecha	Calendario nativo del navegador	search	Búsqueda	Cosmético: botón ✕ para borrar

Validación nativa: atributos clave

Validación = usabilidad, NO seguridad. El servidor debe revalidar SIEMPRE.

required

`required`

El campo no puede quedar vacío

min / max

`min="0" max="100"`

Rango numérico o de fechas

maxlength

`maxlength="80"`

Máximo de caracteres permitidos

autocomplete

`autocomplete="email"`

Sugiere valores guardados por el navegador

pattern

`pattern="[0-9]{10}"`

Regex que el valor debe cumplir

minlength

`minlength="8"`

Mínimo de caracteres exigidos

step

`step="0.01"`

Granularidad de un número o fecha

placeholder

`placeholder="ej: jdoe@..."`

Pista visible cuando el campo está vacío

Demo: formulario de inscripción UTEQ

Sin una sola línea de JavaScript: validaciones del lado cliente listas

```
<form action="/api/inscripcion" method="POST">
  <fieldset>
    <legend>Datos personales</legend>
    <label for="cedula">Cédula:</label>
    <input id="cedula" name="cedula"
      type="text" pattern="[0-9]{10}"
      required>
    <label for="email">Email:</label>
    <input id="email" name="email"
      type="email"
      pattern=".+@uteq\.edu\.ec" required>
    <label for="fnac">Nacimiento:</label>
    <input id="fnac" name="fnac"
      type="date"
      min="1980-01-01" max="2010-12-31"
      required>
    <label for="prom">Promedio:</label>
    <input id="prom" name="prom"
      type="number"
      min="6.0" max="10.0" step="0.01">
  </fieldset>
  <button type="submit">Inscribirme</button>
</form>
```

El navegador valida solo:

- ✓ Cédula con 10 dígitos exactos
- ✓ Email con dominio @uteq.edu.ec
- ✓ Fecha en rango 1980-2010
- ✓ Promedio entre 6.0 y 10.0
- ✓ Todos los required no vacíos

Recuerde:

*El cliente puede ser manipulado.
Toda validación debe duplicarse en el servidor (semanas 5-9).*

BLOQUE 05

Multimedia y APIs

nativas de HTML5

El estándar abierto que reemplazó a Flash y Silverlight en 2020

Las 5 APIs nativas que cambiaron la web

Antes: necesitabas plugins propietarios. Ahora: vienen integrados en todo navegador.



<video> y <audio>

Reproducción multimedia nativa, con controles, subtítulos (<track>) y múltiples formatos.



<canvas>

Lienzo para gráficos 2D y 3D (con WebGL).
Juegos, visualizaciones, gráficos interactivos.



Geolocation API

Acceder a la ubicación geográfica del usuario, previo consentimiento explícito.



Web Storage

localStorage y sessionStorage para guardar datos del lado cliente (5-10 MB).



Drag-and-Drop API

Arrastrar y soltar elementos entre componentes de la página, nativamente.

<video> con accesibilidad completa

WCAG 1.2.2 (subtítulos) y 1.2.5 (audiodescripciones) son exigibles legalmente

```
<figure>
  <video controls width="640"
    preload="metadata"
    poster="poster-uteq.jpg"
    crossorigin="anonymous">

    <!-- WebM primero (Chrome/Firefox) -->
    <source src="bienvenida.webm"
      type="video/webm">
    <!-- MP4 fallback (Safari/Edge) -->
    <source src="bienvenida.mp4"
      type="video/mp4">

    <!-- Subtítulos para usuarios sordos -->
    <track kind="captions"
      src="bienvenida.es.vtt"
      srclang="es" default>
    <!-- Audiodescripciones para usuarios ciegos -->
    <track kind="descriptions"
      src="bienvenida.desc.vtt"
      srclang="es">
  </video>
  <figcaption>Video de bienvenida UTEQ.</figcaption>
</figure>
```

Atributos críticos

controls	Muestra el reproductor nativo
preload	none metadata auto
poster	Imagen previa a reproducir
autoplay	Solo si está muted
loop	Reproducción en bucle
muted	Silenciado por defecto

<track> kind=...

captions → sordos
subtitles → traducción
descriptions → ciegos
chapters → marcadores

Almacenamiento del cliente

Cómo el navegador guarda datos: cuándo elegir cada mecanismo

Mecanismo	Características	Capacidad	API
Cookies	Persisten; se envían al servidor	4 KB	document.cookie
localStorage	Persistente, solo strings	5–10 MB	API síncrona
sessionStorage	Solo dura mientras la pestaña esté abierta	5–10 MB	API síncrona
IndexedDB	Persistente, transaccional, objetos	GB	API asíncrona
Cache API	Guarda Response (Service Workers)	GB	API asíncrona

 **REGLA DE ORO** (la veremos en semana 10): **NUNCA** guarde tokens JWT en localStorage. Cualquier ataque XSS puede leerlos. La solución correcta: cookie **HttpOnly + Secure + SameSite=Strict**.

XML y su lugar en el ecosistema 2026

Dónde sigue vivo XML hoy — y dónde fue reemplazado por JSON, YAML o TOML

Dialectos XML que siguen vivos:

SVG

Gráficos vectoriales: iconos, diagramas, ilustraciones que escalan.

XHTML

HTML reformulado como XML estricto (correos, sistemas legados).

MathML

Notación matemática para sitios académicos.

RSS / Atom

Sindicación de contenido entre sitios y agregadores.

SOAP

Servicios web XML (lo veremos en semana 15).

XSLT

Transformaciones XML hacia otros formatos.

XSLT + XAML + DocBook

Otros usos especializados (transformaciones, interfaces WPF, documentación técnica) que persisten en industrias específicas.

Moraleja para el desarrollador 2026: XML no murió — se especializó. JSON ganó las APIs REST y la web, YAML ganó la configuración DevOps (Docker, Kubernetes), TOML ganó Cargo y pyproject. Pero XML sigue dominando: SVG, SOAP, Maven, AndroidManifest y formatos de oficina.

XML vs JSON vs YAML — el mismo dato

Tres formas de representar exactamente la misma información

XML

```
<estudiante>
  <cedula>0950123456</cedula>
  <nombre>Ana Pérez</nombre>
  <carrera>SW</carrera>
  <activo>true</activo>
  <materias>
    <m>WEB</m>
    <m>BD</m>
  </materias>
</estudiante>
```

Casos de uso 2026

SOAP · Maven
AndroidManifest

JSON

```
{
  "cedula": "0950123456",
  "nombre": "Ana Pérez",
  "carrera": "SW",
  "activo": true,
  "materias": [
    "WEB",
    "BD"
  ]
}
```

Casos de uso 2026

APIs REST · NoSQL
package.json

YAML

```
estudiante:
  cedula: 0950123456
  nombre: "Ana Pérez"
  carrera: SW
  activo: true
  materias:
    - WEB
    - BD
  # comentarios OK
```

Casos de uso 2026

Docker · Kubernetes
GitHub Actions

15 buenas prácticas profesionales

Reglas de oro del HTML profesional — verificadas antes de cada commit

1 Usa `<!DOCTYPE html>` en la primera línea.

4 Cierra todas las etiquetas, incluso las opcionales.

7 Atributos booleanos sin valor (`required`, `disabled`).

10 `loading='lazy'` en imágenes below the fold.

13 Jerarquía de `h1`→`h2`→`h3` sin saltos.

2 `<meta charset='UTF-8'>` dentro de los primeros 1024 bytes.

5 Comillas dobles consistentes en los atributos.

8 Imágenes SIEMPRE con alt descriptivo.

11 `<label>` asociado a cada `<input>` por `for`/`id`.

14 Validar con validator.w3.org/nu/ antes de commit.

3 `<meta viewport>` en toda página mobile-friendly.

6 No abuses de `<div>`: usa elementos semánticos.

9 Imágenes con `width` y `height` (evitan layout shift).

12 autocomplete con valores estándares.

15 Cero CSS inline (excepto en correos electrónicos).

Actividad práctica en clase

Laboratorio: formulario de inscripción UTEQ paso a paso (60 minutos)

1

10 min

Setup

Crear proyecto appweb-formularios en IntelliJ. Archivo registro.html con la plantilla mínima de HTML5.

2

15 min

Estructura

Construir el <form> con <fieldset> y <legend>. Cuatro inputs: cédula, email, fecha, promedio.

3

15 min

Validación

Agregar pattern, min, max, step, required. Probar enviando el formulario vacío.

4

10 min

Móvil

Abrir DevTools, modo dispositivo móvil. Tocar cada campo y verificar el teclado virtual.

5

10 min

Validar

Subir a validator.w3.org/nu/. Cero errores. Auditar con axe DevTools.

Tarea autónoma · Ejercicio 2.2

Portafolio profesional accesible — entrega individual por GitHub

Construir `portafolio.html` que demuestre dominio integral de HTML5 semántico, validación, multimedia y accesibilidad.

Requisitos obligatorios



Estructura semántica completa: `<header>`, `<nav>`, `<main>`, `<article>` (mínimo 3), `<aside>`, `<footer>`.



Un `<video>` con al menos un `<track kind="captions">` (incluir el archivo `.vtt`).



Un `<details>+<summary>` para una sección de FAQ.



Al menos un `<figure>+<figcaption>` con `<picture>` sirviendo múltiples resoluciones.



Formulario de contacto con 6 tipos distintos: text, email, tel, date, number, url.



Una `<table>` con `<caption>`, `<thead>`, `<tbody>` y `<th scope>`.

Criterios de aceptación: cero errores en W3C Validator · cero violaciones críticas en axe DevTools · navegación 100% por teclado.

Entrega: archivo HTML + screenshots de ambos validadores, en repositorio Git.

Fecha límite: antes de la sesión de la semana 03 · Ponderación: TA02 (5%).

Herramientas de validación profesional

Tres puertas que todo HTML profesional debe pasar antes de un commit



W3C Markup Validator

validator.w3.org/nu/

Valida que tu HTML cumpla la especificación oficial. Detecta:

- Etiquetas no cerradas o anidadas mal
- Atributos inválidos o duplicados
- IDs repetidos
- DOCTYPE incorrecto



axe DevTools

Chrome / Firefox extension

Audita accesibilidad WCAG 2.1. Detecta:

- Imágenes sin alt
- Contrastes insuficientes
- Formularios sin label
- Jerarquías de heading rotas



Lighthouse

DevTools (F12)

Auditoría integral del sitio. Mide:

- Performance
- Accesibilidad
- SEO
- Best practices

Errores frecuentes — qué evitar

Lo que más penalizamos en los entregables de prácticas anteriores

- | | | |
|---|--|--|
| ✗ | Usar <code><section></code> sin <code><h2>...<h6></code> dentro. | → usar <code><div></code> o agregar el encabezado. |
| ✗ | Múltiples <code><main></code> en una sola página. | → solo uno por documento. |
| ✗ | <code></code> sin atributo <code>alt</code> . | → siempre <code>alt</code> descriptivo o <code>alt=""</code> si es decorativa. |
| ✗ | <code><input></code> sin <code><label for="id"></code> . | → los lectores de pantalla no podrán anunciarlo. |
| ✗ | Saltos de jerarquía: <code>h1</code> → <code>h3</code> . | → respetar la secuencia <code>h1</code> → <code>h2</code> → <code>h3</code> . |
| ✗ | <code>
</code> para crear espacios entre párrafos. | → usar <code>margin/padding</code> de CSS. |
| ✗ | Dos elementos con el mismo <code>id="..."</code> . | → los IDs son únicos por documento. |
| ✗ | Tablas para maquetar el layout. | → usar <code>Flexbox</code> o <code>Grid</code> (semana 3). |
| ✗ | Validar solo en el cliente con HTML5. | → revalidar siempre en el servidor. |

Mini-quiz de cierre

¿Estás listo para la sesión de la próxima semana? Responde mentalmente.

P: ¿Qué etiqueta uso para un comentario de foro publicable en RSS?

R: `<article>`

P: ¿Cuántos `<main>` puede haber por página?

R: *Exactamente uno.*

P: ¿Dónde NO debo guardar un JWT?

R: *En localStorage: cualquier XSS lo roba.*

P: ¿Por qué `<meta charset="UTF-8">` debe ir en los primeros 1024 bytes?

R: *Para no corromper las tildes y la ñ durante el parsing.*

P: ¿Qué hace `pattern="[0-9]{10}"`?

R: *Exige que el valor sean 10 dígitos exactos (ej: cédula ecuatoriana).*

P: ¿Cuándo uso `<section>` en lugar de `<div>`?

R: *Cuando agrupo contenido temático CON encabezado.*

Bibliografía y recursos

Lectura previa a la próxima sesión y referencias para el portafolio

Libros recomendados

Robbins, J. N. (2018). *Learning Web Design* (5ª ed.). O'Reilly.

Duckett, J. (2014). *HTML & CSS: Design and Build Web Sites*. Wiley.

Lawson, B. & Sharp, R. (2011). *Introducing HTML5* (2ª ed.). New Riders.

Nixon, R. (2021). *Learning PHP, MySQL & JavaScript* (6ª ed.). O'Reilly.

Hickson, I. & Berjon, R. (2014). *HTML5 Recommendation*. W3C.

WHATWG (vigente). *HTML Living Standard*.

Recursos en línea

- MDN Web Docs developer.mozilla.org/es/docs/Web/HTML
- W3C Validator validator.w3.org/nu/
- axe DevTools deque.com/axe/devtools
- WCAG 2.1 Quick Ref w3.org/WAI/WCAG21/quickref/
- HTML Living Standard html.spec.whatwg.org
- Can I Use caniuse.com
- HTML5 Doctor html5doctor.com
- WebAIM webaim.org

Hasta la próxima semana

Semana 03 — CSS para el diseño y presentación web

Lo que llevas hoy:

- ✓ Construir HTML5 estructuralmente correcto
- ✓ Aplicar semántica de header / nav / main / article / section / aside / footer
- ✓ Diseñar formularios con validación nativa sin JS
- ✓ Integrar multimedia accesible (subtítulos, audiodescripciones)
- ✓ Conocer el lugar de XML y las alternativas modernas (JSON, YAML)